

Problem 1: Self assessment.

For problem 6 (a). I feel my solution is better because in the solution they have a for loop that loops n times, but since n is an integer it is size $\log(n)$ in terms of the input size, but they do a $O(n)$ operation, so if n is really large it could be exponential in terms of the input. Thus, to fix this in my solution I checked if $n \geq |C|$, and if it was all you had to check if each $c_i \in C$ was less than the workload limit k .

Problem 2: Uniform sampling with constant memory.

```

1: function UNIFORM( $a_1, \dots, a_n$ )
2:   for  $i = 1, \dots, n$  do
3:     Overwrite the current memory with  $a_i$  with probability  $1/i$ 
4:     Otherwise, the memory stays unchanged (with probability  $1 - 1/i$ )
5:   Output the item in the memory

```

Prove that UNIFORM selects a uniformly random sample from the sequence $(a_1, \dots, a_n)$. In other words, show that each element a_i has an equal probability $1/n$ being in the memory after processing all items.

Proof.

Let $\mathbb{P}(a_i)$ be the probability that a_i is chosen at time i , and let $\mathbb{P}(\neg a_j)$ be the probability that a_j was not chosen at time j .

Thus, $\mathbb{P}(a_i \text{ is the output}) = \mathbb{P}(a_i \cap \neg a_{i+1} \cap \dots \cap \neg a_n)$ since all of these events are only dependent on time t . This implies that they are all independent of each other.

Therefore,

$$\begin{aligned}
 \mathbb{P}(a_i \text{ is the output}) &= \\
 &= \mathbb{P}(a_i \cap \neg a_{i+1} \cap \dots \cap \neg a_n) \\
 &= \mathbb{P}(a_i) \mathbb{P}(\neg a_{i+1}) \dots \mathbb{P}(\neg a_n) \\
 &= \frac{1}{i} \prod_{k=i+1}^n \left(1 - \frac{1}{k}\right) \\
 &= \frac{1}{i} \prod_{k=i+1}^n \left(\frac{k-1}{k}\right) = \frac{1}{i} \cdot \frac{i}{n} = \frac{1}{n}
 \end{aligned}$$

Therefore, each a_i has an equal probability $1/n$ being the output. ■

Problem 3: Recording record-breaking moments.

The *Hall of Fame* has only *one seat*, reserved for the current fastest time. Every time a record is broken, i.e., $t_i = \min\{t_1, \dots, t_i\}$, the seat changes hands to the new record-holder. Lambda, who is responsible for updating the Hall of Fame, starts to wonder how often they will have to make these updates.

For simplicity, you may assume all t_i 's are *distinct* in this problem.

(a)

Define an indicator random variable that is suitable for this problem. When defining an indicator random variable, be sure to:

1. clearly describe the event for which the variable equals 1 and 0 (you may write "0 otherwise" to be concise), and
2. derive the probability that the indicator random variable equals 1.

Proof.

Let X_i be the random variable s.t. X_i is 1 if t_i is the minimum time of $\{t_1, \dots, t_i\}$ and 0 otherwise.

$\mathbb{P}(X_i = 1) = \mathbb{P}(t_i = \min\{t_1, \dots, t_i\}) = \frac{1}{i}$. Since all times are distinct and in a random order, every permutation of the times is equally likely thus since there is only one minimum in the first i times this implies the probability is $1/i$. ■

(b)

Let X be the number of record-breaking moments. Show that

$$\mathbb{E}[X] = 1 + \frac{1}{2} + \dots + \frac{1}{n}$$

Proof.

Since X is the number of record-breaking moments this happens exactly when t_i is the minimum of all the previous times. Thus, this is exactly when $X_i = 1$.

Therefore, $X = X_1 + \dots + X_n$.

Thus,

$$\mathbb{E}[X] = \mathbb{E}[X_1 + \dots + X_n] = \mathbb{E}[X_1] + \dots + \mathbb{E}[X_n] = \mathbb{P}(X_1 = 1) + \dots + \mathbb{P}(X_n = 1) = 1 + \frac{1}{2} + \dots + \frac{1}{n}. \quad \blacksquare$$

Problem 4: Equi-satisfiability.

In this problem, we define a clause to be in *4-disjunctive-unique-form* (4DUF) if it has *exactly 4 literals* involving *4 distinct variables*, connected by disjunctions ($\vee$).

Given a truth assignment, we say that 4DUF clause is *equi-satisfied* if it contains *exactly two* true literals and *exactly two* false literals.

Now, consider a collection of m 4DUF clauses $C = \{c_1, \dots, c_m\}$. While a variable may appear in multiple clauses, no clause contains repeated variables. Let RAND-ASSIGN be the same randomized algorithm as in lecture: assign random truth values (true or false) to the formula's variables, uniformly and independently. That is, for each variable, assign it to be true or false each with $1/2$ probability, independently of all the others.

(a)

Show that RAND-ASSIGN equi-satisfies a constant fraction $\rho \in [0, 1]$ of the clauses of C in expectation. Analyze and determine the exact value of ρ .

Proof.

Let X be the number of equi-satisfied clauses of C .

Let X_i be 1 if clause $c_i \in C$ is equi-satisfied and 0 if not.

Thus, $X = X_1 + \dots + X_m$.

Therefore, $\mathbb{E}[X] = \mathbb{E}[X_1] + \dots + \mathbb{E}[X_m] = \mathbb{P}(X_1 = 1) + \dots + \mathbb{P}(X_m = 1)$.

Since each variable in clause c_i has a uniform probability of being true or false. This implies that the probability of being equi-satisfied is.

Total outcomes: $2^4 = 16$ for all possible truth assignments

equi-satisfied: The number of ways to choose exactly 2 out of the 4 to be true is $\binom{4}{2} = 6$.

Thus, $\mathbb{P}(X_i = 1) = \frac{6}{16} = \frac{3}{8}$.

Therefore, $\mathbb{E}[X] = m \frac{3}{8}$

Thus, $\rho = \mathbb{E}[X] / |C| = 3/8$ ■

(b)

Using Markov's inequality, give a **lower bound** on the probability that RAND-ASSIGN equi-satisfies at least $\rho/2$ -fraction of the clauses.

Proof.

Let X be the number of equi-satisfied clauses of C . Let $Y = m - X$ which is the number of not equi-satisfied clauses.

Thus by Markov's inequality,

$$\mathbb{P}(Y \geq m(1 - \rho/2)) \leq \frac{\mathbb{E}[Y]}{m(1 - \rho/2)} = \frac{\mathbb{E}[m - X]}{m(1 - \rho/2)} = \frac{m(1 - \rho)}{m(1 - \rho/2)} = \frac{1 - \rho}{1 - \rho/2}.$$

Also since if $X \leq \rho/2 \cdot m$ then

$$Y = m - X > m - \rho/2m = m(1 - \rho/2) \implies \mathbb{P}(X < \rho/2m) \leq \mathbb{P}(Y \geq m(1 - \rho/2))$$

$$\text{Thus, } \mathbb{P}(X < \rho/2 \cdot m) \leq \frac{1 - \rho}{1 - \rho/2}$$

Since $\mathbb{P}(X < \rho/2 \cdot m) = 1 - \mathbb{P}(X \geq \rho/2 \cdot m)$, this implies that $\mathbb{P}(X \geq \rho/2 \cdot m) \geq 1 - \frac{1 - \rho}{1 - \rho/2}$.

Since $\rho = 3/8$. This implies that $\mathbb{P}(X \geq 3/16 \cdot m) \geq 1 - \frac{10}{13} = \frac{3}{13}$ ■

Problem 5:

Let S be a set of n orderable elements, which are distinct without loss of generality. We say that $a \in S$ has *rank* $\text{rank}(a) = k$ if a is the k th smallest element in S . Given input S , the goal is to find a *near median* of S , which is an element having rank between $2n/5 + 1$ and $3n/5$ (inclusive). For simplicity, we assume that n is divisible by 5 (because we can “pad” the set with up to four extra elements).

(a)

Consider the simplest randomized algorithm: output a uniformly random element $a \in S$.

Prove that this algorithm outputs a near median of S with probability at least $1/5$.

Proof.

Since there are $\frac{3n}{5} - \frac{2n}{5} - 1 + 1 = \frac{n}{5}$ elements between $[2n/5 + 1, 3n/5]$.

This implies there are $n/5$ near medians.

Therefore, since each element has a uniform chance of being picked this implies that there is a $(n/5)/(n) = 1/5$ probability of a being a near median. ■

(b)

Briefly describe an $O(n)$ -time algorithm that, given S and an element $a \in S$, determines whether a is a near median.

Proof.

Just traverse the set S and count the number of elements s.t. $b < a$ for $b \in S$ call this number k . Thus, $\text{rank}(a) = k + 1$ since k elements are smaller than a .

Now if $k + 1 \in [2n/5 + 1, 3n/5]$ then a is near median otherwise it is not.

This is $O(n)$ since we just traverse the array which has n things in it, and the rest are $O(1)$ operations.

This is correct since we defined near median to be if the rank is in the set $[2n/5 + 1, 3n/5]$. ■

(c)

Give an $O(n)$ -time randomized algorithm that, on input S , outputs a near median with probability at least 99%.

Proof.

Since the probability of picking a near median is $n/5$ this implies the probability of not picking a near median is $(4/5)$. Thus, if we randomly pick k times we would have a probability of $(4/5)^k$ of not getting a near median once.

Thus, for the failure to be at most 1% we get that $(4/5)^k \leq 0.01 \implies k \log(4/5) \leq \log(0.01) \implies k \geq 20.7$ Thus, let $k = 21$.

Thus for $i = 1$ to 21 pick an element a from S then run the algorithm from b on it to check if it is near median. If it is then return a and halt.

If we found no near medians for all of the 21 iterations then just output the most recent pick.

Runtime:

This is $O(21n) = O(n)$.

Correctness:

Since we have show that the chance of not picking a near median in 21 trials is less than 1%. This implies our algorithm with output a near median with probability of at least 99%. ■

Problem 6: Random binary search trees.

For a random binary search tree on $1, \dots, n$, prove that the expected depth of the node with any particular value is $O(\log n)$. In other words, for any particular value in a random binary search tree, searching for it takes $O(\log n)$ time.

Proof.

Let $v \in \{1, \dots, n\}$.

For each $j \neq v$ $X_j = 1$ if j is an ancestor of v , and 0 otherwise.

Thus, for X being the depth of v is equal to $X = \sum_{j \neq v} X_j$.

Therefore, $\mathbb{E}[X] = \sum_{j \neq v} \mathbb{E}[X_j] = \sum_{j \neq v} \mathbb{P}(X_j = 1)$.

If $j < v$. For j to be an ancestor of v , it must be the first element chosen among all the elements in the interval $\{j, \dots, v\}$. Since if j is picked first then j becomes the root to all elements in $\{j + 1, \dots, v\}$.

If j is not picked first and lets say $k \in \{j + 1, \dots, v - 1\}$ is then this would mean that k would become the root, and we would split the tree s.t. $\{j, \dots, k - 1\}$ is in the left subtree and $\{k + 1, \dots, v\}$ are in the right subtree. Thus, j would not be an ancestor of v .

Thus, the probability of picking j first out of this set is $\frac{1}{v - j + 1}$ since we pick each element with uniform probability.

Thus, using the similar argument for $j > v$ we get $\frac{1}{j - v + 1}$ probability of j being an ancestor.

Therefore,

$$\begin{aligned} \mathbb{E}[X] &= \sum_{j=1}^{v-1} \frac{1}{v-j+1} + \sum_{j=v+1}^n \frac{1}{j-v+1} \\ &\leq \sum_{k=1}^{v-1} 1/k + \sum_{k=1}^n 1/n \\ &\leq 2\ln(n) + 2 = O(\log n) \end{aligned}$$

Therefore, the expected depth of $v \in \{1, \dots, n\}$ is $O(\log n)$ ■

Problem 7: Randomized vertex cover.

In this problem, we will consider randomized algorithms for approximating vertex-cover problems. Recall that the “double cover” algorithm *deterministically* gives a 2-approximation to the minimum vertex cover problem, but in this problem you will show that a randomized approach is more versatile.

(a)

Consider a probability experiment with set S with k distinct items. In each round:

- With probability **at least** p (where $p \in (0, 1]$), one *new* item (not previously chosen) is selected.
- Otherwise, no item is selected on that round.
- The experiment ends when all k items are selected.

Let R be a random variable denoting the number of rounds until the experiment ends. Prove that

$$\mathbb{E}[R] \leq \frac{k}{p}$$

Proof.

For each $i = 1, \dots, k$, define R_i as the number of rounds needed to select the i -th new item after $i - 1$ items have been chosen.

Since the experiment finishes when all k items are chosen, the total number of rounds R to complete the experiment can be expressed as

$$R = R_1 + \dots + R_k$$

Since R_i is the number of rounds until one success of picking an item. It is a geometric random variable. Also, since the probability of picking one item is greater than or equal to p this implies that $\mathbb{P}(R_1 = i) \leq (1 - p)^{i-1}p$.

Also since the probability of picking an item after picking j items or picking i items this implies that the distributions are the same. Therefore, $\mathbb{P}[R_i = c] = \mathbb{P}[R_j = c] \implies \mathbb{E}[R_i] = \mathbb{E}[R_j]$ for all i, j .

Therefore,

$$\begin{aligned}
 \mathbb{E}[R] &= \sum_{i=1}^k \mathbb{E}[R_i] = k\mathbb{E}[R_1] \\
 &= k \sum_{i=1}^{\infty} i \mathbb{P}[R_1 = i] \\
 &\leq k \sum_{i=1}^{\infty} i (1-p)^{i-1} p \\
 &= \frac{kp}{(1 - 1 + p)^2} = \frac{k}{p}
 \end{aligned}$$

(b)

Consider the following algorithm

```

1: function RANDOMSINGLECOVER( $G = (V, E)$ )
2:    $C \leftarrow \emptyset$ 
3:   while there is some edge  $e = (u, v)$  that is not covered by  $C$  do
4:     Add exactly one of  $u$  or  $v$  to  $C$ , each with probability  $1/2$ 
5:   return  $C$ 

```

Prove that RANDOMSINGLECOVER outputs a 2-approximation, in expectation, for the minimum vertex cover problem. In other words, show that

$$\mathbb{E}[|C|] \leq 2 \cdot |C^*|$$

where $C^* = \{v_1, \dots, v_k\}$ is some minimum vertex cover in the input graph.

Proof.

Let

$$S_{\text{chosen}} \subseteq C^*$$

denote the set of vertices from the minimum vertex cover C^* that have been “hit” (i.e., chosen by the probability experiment) up to the current iteration. We show by induction that after every iteration,

$$S_{\text{chosen}} \subseteq C,$$

where C is the set of vertices chosen by the algorithm.

Induction:

Base Case:

Before any iteration, the algorithm initializes $C = \emptyset$. Correspondingly, no vertex from C^* has been chosen by the experiment yet, so $S_{\text{chosen}} = \emptyset$. Clearly, $\emptyset \subseteq \emptyset$.

Inductive Step:

Assume that after some iteration the invariant holds, that is, $S_{\text{chosen}} \subseteq C$. Now consider an iteration where the algorithm processes an edge $e = (u, v)$ that is currently uncovered by C . Because C already contains all previously chosen vertices in C^* (by the inductive hypothesis), neither u nor v is in S_{chosen} . Since C^* is a vertex cover, it must cover every edge in the graph, so the edge e must satisfy

$$\{u, v\} \cap (C^* \setminus S_{\text{chosen}}) \neq \emptyset.$$

Upon processing e , the algorithm adds one of its endpoints with probability $1/2$. Hence, with probability at least $1/2$ (or even higher if both endpoints belong to $C^* \setminus S_{\text{chosen}}$), the chosen vertex belongs to $C^* \setminus S_{\text{chosen}}$. In that case, it is immediately included in C , and we can consider it as being added to S_{chosen} with the ongoing experiment. If we did not pick the vertex that was in C^* then this is associated to in the experiment when we do not choose an item, so S_{chosen} remains the same thus $C^* \setminus S_{\text{chosen}}$.

Next, we show that the algorithm terminates by the time the probability experiment terminates.

Suppose, for contradiction, that the probability experiment has terminated that is, but the algorithm has not yet terminated. Then there exists at least one edge $e = (u, v)$ that remains uncovered by C .

However, since we have maintained the invariant $S_{\text{chosen}} \subseteq C$ and $S_{\text{chosen}} = C^*$, it follows that every vertex in C^* is also in C . Because C^* is a vertex cover, every edge in the graph must be covered by some vertex in C^* . Therefore, the edge e must be covered by C , which is a contradiction. This implies that if the experiment terminates, the algorithm must also have terminated. In other words, the algorithm halts no later than the experiment does.

Each iteration of the algorithm corresponds exactly to one round of the probability experiment, with one vertex being added to C per iteration. Let R denote the total number of rounds (or iterations) performed by the algorithm. From the analysis in part (a) of the problem, we know that if an experiment selects one new vertex from C^* in each round with a success probability of at least $1/2$, then the expected number of rounds needed to select all $|C^*|$ vertices is at most

$$\mathbb{E}[R] = \mathbb{E}[|C|] \leq \frac{k}{\frac{1}{2}} = 2k.$$

Thus, the algorithm is a 2-approximation in expectation for the minimum vertex cover problem. ■

(c)

For each of the following algorithms, show that it can yield an *arbitrarily large* approximation ratio for this problem, by describing an explicit graph and showing how the (expected) weight of the algorithm's output cover can be very large relative to the optimal weight.

- (i) The double-cover algorithm from lecture.
- (ii) The RANDOMSINGLECOVER algorithm from ??.
- (iii) The following greedy algorithm:

```

1: function GREEDY( $G = (V, E, w)$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has an edge do
4:     Remove all isolated vertices of  $G$ 
5:     Add a lowest-weight vertex in  $G$  to  $S$  and remove all its incident edges
6:   return  $S$ 

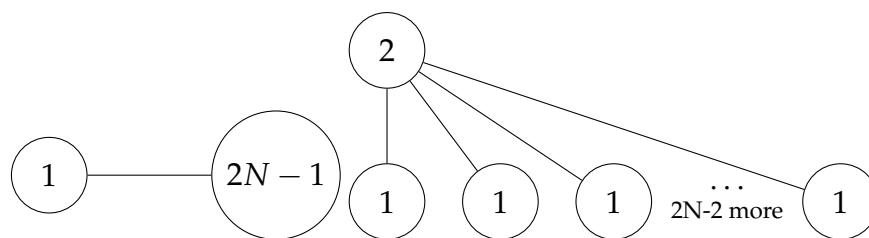
```

Proof.

Let X be the weight of the algorithm's vertex cover.

Fix $N > 1 \in \mathbb{N}$.

Consider the graphs



The optimal vertex cover for the graph to the left is to just pick the vertex with weight 1.

The optimal vertex cover for the graph to the right is to just pick the vertex with weight 2.

- (i) Look at the graph with two vertices. Since there is only one edge and the algorithm always picks both vertices this implies that $\mathbb{E}[X] = 2N \leq 2NOPT = 2N$

- (ii) Look at the graph with two vertices. Since there is only one edge and each vertex has an equal chance of being picked this implies that $\mathbb{E}[X] = (1/2)1 + (1/2)(2N - 1) = N \leq \text{NOPT} = N$
- (iii) Look at the graph with $2N + 1$ vertices. By definition of the algorithm it will add all the ones to our cover first, but then since all of the ones are connected to 2 it will disconnect 2 so we will be done. Thus, we will have a total weight of $2N$. Thus, $\mathbb{E}[X] = 2N \leq \text{NOPT} = 2N$.

Therefore, for all of these algorithms there is a graph that will yield an arbitrarily large approximation ratio. ■

(d)

Proof.

Consider the following algorithm let $w(v)$ be the weight of vertex v .

```

1: function RANDOMWEIGHTCOVER( $G = (V, E)$ )
2:    $C \leftarrow \emptyset$ 
3:   while there is some edge  $e = (u, v)$  that is not covered by  $C$  do
4:     Add exactly one of the vertices, and pick  $u$  with probability  $\frac{w(v)}{w(v) + w(u)}$ 
5:   return Sum of  $C$ 

```

Let C^* be an optimal vertex cover and let $a \in C^*$. First partition E s.t. each edge is assigned to a vertex in C^* . We can do this since C^* is a vertex cover, so every edge is connected. We want to do this because we only want to look at each edge once, and it is easier to look locally at vertices in C^* .

Consider $P(a) \subset E$ all of the edges that are assigned to a . For each $e \in P(a)$ since our algorithm's output is a vertex cover C this means that at most one vertex of e was added to C . Therefore, let V_a be the set of vertices added by the algorithm that are connected to $P(a)$.

Induction: We will induct on the size of $P(a)$, and show that $\mathbb{E}[\sum_{v \in V_a} w(v)] \leq 2w(a)$.

Base Case: $k = 0$

Then this implies that either $V_a = \emptyset$. Thus, $\mathbb{E}[\sum_{v \in V_a} w(v)] = 0 \leq 2w(a)$.

Inductive Step: Assume that for all $k \leq n$: Consider $k = n + 1$.

First, consider $v' \in V_a$. Thus, $(a, v') \in P(a)$, and

$$E\left[\sum_{v \in V_a} w(v)\right] = \mathbb{P}(v' \text{ being picked})[w(v') + E\left[\sum_{v \in V_a \setminus \{v'\}} w(v)\right]] + \mathbb{P}(a \text{ being picked})w(a)$$

The reason why this is correct because in case 1 our algorithm picked v' , and thus we have to also account for the rest of the edges in $P(a)$, but in the other case if we pick a then we are done since we will now cover all of $P(a)$

$$= \frac{w(a)}{w(a) + w(v')}[w(v') + E\left[\sum_{v \in V_a \setminus \{v'\}} w(v)\right]] + \frac{w(v')}{w(a) + w(v')}w(a)$$

Thus, by induction hypothesis since $V_a \setminus v'$ is size n we get that

$$\leq 2 \frac{w(a)w(v')}{w(a) + w(v')} + 2 \frac{w(a)^2}{w(a) + w(v')} = 2w(a) \frac{w(v') + w(a)}{w(a) + w(v')} = 2w(a)$$

Therefore, $E[\sum_{v \in C} w(v)] = E[\sum_{a \in C^*} \sum_{v \in V_a} w(v)]$

The reason this is true is because the V_a 's form a partition because we assign each edge in the graph uniquely to a vertex in the optimal vertex cover C^* , ensuring that every edge is covered exactly once. Specifically, for each vertex $a \in C^*$ we define $P(a)$ as the set of edges assigned to a . Since every edge is incident to at least one vertex in C^* , and we assign it to just one vertex, the $P(a)$'s are disjoint. Then, for each a , the set V_a is defined as all vertices chosen by the algorithm that are incident to at least one edge in $P(a)$. As a result, every vertex added by the algorithm is uniquely associated with one of the $P(a)$'s ensuring that each such vertex is counted in exactly one V_a . Therefore, the collection $\{V_a : a \in C^*\}$ forms a partition of the set of vertices selected by the algorithm. Thus, $E[\sum_{a \in C^*} \sum_{v \in V_a} w(v)] = \sum_{a \in C^*} E[\sum_{v \in V_a} w(v)] \leq \sum_{a \in C^*} 2w(a) = 2\text{OPT}$. ■